androsec

EXPLOITATION REPORT

AWS access key (format match)

owasp.sat.agoat

Package owasp.sat.agoat

Severity Critical

CVSS band: Critical; partial dynamic evidence

Generated 2026-11-14 16:41 UTC

Disclaimer

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.

Table of contents

Assessment target 3

Executive summary 4

Assessment write-up 5

Captured evidence 10

Disclaimer 17

Cover shows a qualitative CVSS severity band only — not a scored CVSS 3.1 vector. A real base score requires Attack Vector / Complexity / Privileges / UI / Scope / CIA.

Assessment target

Application and device used for this verification.

App name	owasp.sat.agoat
Package	owasp.sat.agoat
Version	1.0
Min / Target SDK	19 / 33
APK size	6.8 MB
SHA-256	3e6f6b538b82874dd94c0d8cdadd69ba54d49b59d2c7eb055073892e8a4fc0e0
Device	VirtualBox
Android	9
Serial	10.0.2.5:5555
Report ID	exploitation_20261114_164023
Generated	2026-11-14 16:41 UTC
This finding	SECRET_AWS_KEY — AWS access key (format match)
Finding severity	Critical
CVSS	CVSS band: Critical; partial dynamic evidence
Dynamic verification	PARTIAL

Executive summary

Single-finding exploitation report — not a full-application scorecard.

Finding severity **Critical** — **CVSS band: Critical; partial dynamic evidence** — dynamic verification **PARTIAL** — confidence **Medium**

Critical AWS access key (format match)

RULE	SECRET_AWS_KEY
SEVERITY	Critical
CVSS	CVSS band: Critical; partial dynamic evidence
CONFIDENCE	Medium
MASVS	MASVS-STORAGE-1
VERIFICATION	PARTIAL
EVIDENCE	`AKIAX56QKKOLPQ7G7ABC [format-only — not validated against AWS]`
RECOMMENDATION	Rotate if real; never ship cloud credentials. Treat lab placeholders as training signals.

Assessment write-up

Narrative produced for this finding, including scope, methodology, evidence, and remediation.

MSS VAPT Exploitation Write-Up

Executive Summary

Dynamic verification: PARTIAL.

This report documents a single finding — **SECRET_AWS_KEY: Hardcoded AWS Access Key (format match)** — identified within the Android application `owasp.sat.agoat` (v1.0) during dynamic verification activities performed against a VirtualBox-hosted Android 9 device (10.0.2.5:5555). A string matching the AWS Access Key ID format (`AKIAX56QKKOLPQ7G7ABC`) was located inside the application's compiled `classes2.dex` resource after extracting the installed APK from the device. This value has **not** been validated against live AWS infrastructure; it is reported strictly as a format-match static artifact discovered via a dynamic extraction workflow.

CVSS band: Critical; partial dynamic evidence. The dynamic verification gate for this finding returned a **PARTIAL** confirmation status — no expected proof tokens from the finding definition were matched against the exploit command output stream in a manner sufficient to upgrade this finding to CONFIRMED. Accordingly, this finding is reported as **Critical** severity with **Medium** confidence, and all claims below are scoped strictly to what was observed in the Executed steps. No AWS credential validation, cloud resource enumeration, or secret-abuse was performed or claimed.

This document presents only the single finding described above. No application-wide risk scores or severity distribution totals are referenced in this report.

Scope & Authorization

- **Target application:** `owasp.sat.agoat` (package: `owasp.sat.agoat`), version 1.0
- **Target device:** VirtualBox-hosted Android 9 emulator/device, accessible via ADB at `10.0.2.5:5555`
- **Testing window:** Authorized graduation-project VAPT exercise against an intentionally vulnerable training application (AGoat)
- **Authorization basis:** This assessment was performed under explicit authorization for academic/ graduation-project purposes against a designated test instance. No production systems, real AWS accounts, or third-party infrastructure were targeted or accessed.
- **Out of scope:** Validation or use of the discovered key against live AWS APIs; any cloud-side enumeration; any other findings from the broader application assessment.

OWASP Mobile Top 10 Classification

- **Primary category:** M9 — Insecure Data Storage (OWASP MASVS-STORAGE-1: hardcoded secrets shipped within the application binary)
- **Secondary category:** M10 — Extraneous Functionality / Hardcoded Secrets (credentials embedded in compiled code, retrievable via static extraction of a dynamically pulled APK)

Environment & Topology

COMPONENT	DETAIL
Target App	owasp.sat.agoat v1.0
Device	VirtualBox / Android 9
ADB Endpoint	10.0.2.5:5555
Extraction Host	Local workstation with `adb`, `unzip`, `grep` tooling
Access Method	ADB over TCP to emulator instance

The device was accessed over a standard ADB TCP connection. The application package was resolved via `pm path`, pulled locally as an APK, and unpacked for static string inspection as part of the dynamic verification workflow.

Methodology

1. Establish ADB connectivity to the target device and wake/unlock the screen state.
2. Launch the target application via the Android activity manager (`monkey`) to confirm the package is installed, resolvable, and active.
3. Resolve the installed APK path via `pm path` and pull the binary to a local analysis host.
4. Extract the APK archive and perform string-pattern searching (`grep -ra AKIA`) across extracted contents to identify AWS Access Key ID–formatted strings.
5. Capture screenshots as supplementary visual state evidence at relevant steps.
6. Enumerate package components (`dumpsys package`) to document the application's exposed attack surface context.
7. Evaluate command output against the expected proof-token gate for this finding to determine confirmation level.

Finding Detail

- **Finding:** SECRET_AWS_KEY — AWS Access Key (format match)
- **Severity:** Critical
- **CVSS band: Critical; partial dynamic evidence**
- **Confidence:** Medium
- **Affected Component:** `classes2.dex` within the application's `base.apk`
- **Description:** A string matching the AWS Access Key ID format — `AKIAX56QKKOLPQ7G7ABC` — was located during static inspection of the application binary pulled directly from the live device. This indicates the application package, as physically installed and distributed, embeds a credential-shaped secret within its compiled code (MASVS-STORAGE-1). The secret was **not** validated against AWS; this finding reflects format-match detection only.
- **Verification Status:** PARTIAL. The exploitation gate did not match expected proof tokens against command output with sufficient confidence to mark this CONFIRMED. The presence of the string was observed within raw extracted binary output, but no additional validation (e.g., live credential test, secret-usage confirmation within app runtime behavior) was performed or is claimed.

- **Impact (if validated further):** Should this key correspond to a live, active AWS credential, exposure could enable unauthorized access to AWS resources accessible by the associated IAM principal, including potential data exfiltration, resource abuse, or lateral movement within connected cloud infrastructure. This impact is **conditional** and unverified in this engagement.

Exploitation / Verification Steps

Step 1 — Wake device

- Command: `adb shell input keyevent KEYCODE_WAKEUP`
- Status: NO USABLE OUTPUT / FAILED

```
OK (exit 0, no output)
```

Step 2 — Dismiss keyguard if present

- Command: `adb shell wm dismiss-keyguard`
- Status: NO USABLE OUTPUT / FAILED

```
OK (exit 0, no output)
```

Step 3 — Launch target application

- Command: `adb shell monkey -p owasp.sat.agoat -c android.intent.category.LAUNCHER 1`
- Status: PRODUCED REAL OUTPUT

```
bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=2ms (0ms mobile, 0ms wifi, 2ms not connected)
```

Step 4 — Pull installed APK and search for AWS key string

- Command: `adb pull $(adb shell pm path owasp.sat.agoat | tr -d '\r' | cut -d: -f2) /tmp/agoat.apk && unzip -o -q /tmp/agoat.apk -d /tmp/agoat_extracted && grep -ra AKIA /tmp/agoat_extracted`
- Status: PRODUCED REAL OUTPUT

```
/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==/base.apk: 1 file pulled, 0 skipped. 154.6 MB/s
(7100535 bytes in 0.044s)
/tmp/agoat_extracted/classes2.dex: ... AKIAX56QKKOLPQ7G7ABC ...
```

(Output truncated for readability; full raw output contains the string `AKIAX56QKKOLPQ7G7ABC` embedded among unrelated compiled symbol names within `classes2.dex`.)

Step 5 — Post-extraction screenshot

- Command: `screenshot`
- Status: PRODUCED REAL OUTPUT

```
Screenshot saved as screenshot_01.png (role=post_proof)
```

Step 6 — Bring target app to foreground

- Command: `adb shell monkey -p owasp.sat.agoat -c android.intent.category.LAUNCHER 1`
- Status: PRODUCED REAL OUTPUT

```
bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=5ms (0ms mobile, 0ms wifi, 5ms not connected)
```

Step 7 — Dump package components

- Command: `adb shell dumpsys package owasp.sat.agoat`
- Status: PRODUCED REAL OUTPUT

```
Activity Resolver Table:
  Schemes:
      androgoat:
        4ea72a0 owasp.sat.agoat/.AccessControl1ViewActivity filter 6e463ed
          Action: "android.intent.action.VIEW"
          Category: "android.intent.category.DEFAULT"
          Scheme: "androgoat"
          Authority: "vulnapp": -1

  Non-Data Actions:
      android.intent.action.MAIN:
        e7fbe59 owasp.sat.agoat/.SplashActivity filter 892a204
          Action: "android.intent.action.MAIN"
          Category: "android.intent.category.LAUNCHER"
...
```

Step 8 — Post-proof screenshot

- Command: `screenshot`
- Status: PRODUCED REAL OUTPUT

```
Screenshot saved as screenshot_02.png (role=post_proof)
```

Evidence

TYPE	DESCRIPTION	ROLE
Text	Output of `monkey` launch command confirming app launch	proof
Text	Output of APK pull/unzip/grep sequence showing `AKIAX56QKKOLPQ7G7ABC` within `classes2.dex`	proof
Screenshot	`screenshot_01.png` — device state post-extraction	post_proof
Text	Output of `monkey` command bringing app to foreground	proof
Text	Output of `dumpsys package owasp.sat.agoat` showing exposed activity/receiver components	proof

TYPE	DESCRIPTION	ROLE
Screenshot	`screenshot_02.png` — device state post-verification	post_proof

Note: Screenshots are supplementary visual artifacts only; they do not independently confirm the finding without corresponding matched text evidence, per verification policy. The dynamic gate evaluation for this finding returned **PARTIAL**, and this status is preserved throughout this report.

Remediation

1. **Immediately rotate** any AWS IAM credential corresponding to the identified Access Key ID pattern, treating it as potentially compromised regardless of validation status.
2. **Remove all hardcoded credentials** from application source code and compiled resources. Secrets must never be embedded in client-distributed binaries (APKs), as they are trivially recoverable via static extraction, as demonstrated in this report.
3. Adopt a **secure secrets management strategy**, such as retrieving short-lived, scoped credentials from a backend service at runtime (e.g., AWS STS, a secrets manager, or a mobile backend-for-frontend pattern), rather than embedding long-lived static keys.
4. Apply **least-privilege IAM policies** to any credentials used by mobile clients, and enable CloudTrail/credential-usage monitoring to detect anomalous use of rotated keys.
5. Integrate **automated secret-scanning** (e.g., truffleHog, git-secrets, or equivalent CI/CD gates) into the build pipeline to prevent secrets from being compiled into release artifacts in the future.
6. Conduct a full audit of the codebase and build artifacts for other embedded secrets of similar format.

Conclusion

Dynamic verification activities against `owasp.sat.agoat` v1.0 extracted the installed application package directly from a live Android device and identified a string matching the AWS Access Key ID format embedded within compiled application code. This finding is classified as **Critical** severity (**CVSS band: Critical; partial dynamic evidence**) with **Medium** confidence, reflecting that the automated verification gate returned a **PARTIAL** confirmation outcome rather than a fully confirmed result. No credential validation against AWS was performed, and no claims beyond the literal Executed steps and evidence are made in this report. Remediation should proceed under the assumption that the exposed key material requires immediate rotation and that hardcoded secrets must be eliminated from the application binary as a matter of priority.

Captured evidence

Screenshots and command output saved during the run.

Step 1 — Wake device

```
adb shell input keyevent KEYCODE_WAKEUP

OK (exit 0, no output)
```

Step 2 — Dismiss keyguard if present

```
adb shell wm dismiss-keyguard

OK (exit 0, no output)
```

Step 3 — Launch target application

```
adb shell monkey -p owasp.sat.agoat -c android.intent.category.LAUNCHER 1

bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=2ms (0ms mobile, 0ms wifi, 2ms not connected)
```

Step 4 — Pull the installed AGoat APK and search its extracted contents for the hardcoded AWS access key string proving the secret is physically shipped in the shipped binary (MASVS-STORAGE-1 impact: credential exfiltration from any device with the APK).

```
adb pull $(adb shell pm path owasp.sat.agoat | tr -d '\r' | cut -d: -f2) /tmp/agoat.apk && unzip
-o -q /tmp/agoat.apk -d /tmp/agoat_extracted && grep -ra AKIA /tmp/agoat_extracted

/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==/base.apk: 1 file pulled, 0 skipped. 154.6 MB/s
(7100535 bytes in 0.044s)
/tmp/agoat_extracted/classes2.dex:ADD_FROZEN␣
ADD_SUCCESS␣ADD_TILE␣ADJUST_ROW_NUMBER_SKIP␣ADR␣ADR:␣AE␣AES␣AES/CBC/
PKCS7Padding␣AFFECTS_LOGO_MASK␣AGGREGATION_COUNT␣AHT␣AI013103decoder.java␣AI01320xDecoder.java␣AI0
1392xDecoder.java␣AI01393xDecoder.java␣AI013x0x1xDecoder.java␣AI013x0xDecoder.java␣AI01AndOtherAI
s.java␣AI01decoder.java␣AI01weightDecoder.java␣AIChatActivity.kt␣AIHint␣AKIAX56QKKOLPQ7G7ABC␣AL␣
ALIGN_CENTER␣ALIGN_HORIZONTALLY␣
ALIGN_NORMAL␣ALIGN_OPPOSITE␣ALIGN_VERTICALLY␣ALL␣ALLOWED_EAN_EXTENSIONS␣ALLOWED_LENGTHS␣
ALLOW_BINARY␣ALLOW_EDGE_LOCK␣ALLOW_SIZE_IN_UNSPECIFIED_SPEC␣ALLOW_THREAD_GAP_WORK␣ALL_FORMATS␣
ALL_OPERATOR    ALL_PAGES␣ALOG␣ALPHA␣ALPHABET␣ALPHABET_STRING␣
ALPHANUMERIC␣ALPHANUMERIC_CHARS␣ALPHANUMERIC_TABLE␣ALPHA_DISABLED␣ALPHA_DISABLED_LOW␣ALPHA_FRACTIO
N
```

Step 5 — Capture a screenshot of the device state after successful key extraction as supplementary visual evidence.

```
screenshot

Screenshot saved as screenshot_01.png (role=post_proof)
```

Step 6 — Bring target app to foreground

```
adb shell monkey -p owasp.sat.agoat -c android.intent.category.LAUNCHER 1

bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=5ms (0ms mobile, 0ms wifi, 5ms not connected)
```

Step 7 — Dump package components for this app only

```
adb shell dumpsys package owasp.sat.agoat

Activity Resolver Table:
  Schemes:
      androgoat:
        4ea72a0 owasp.sat.agoat/.AccessControl1ViewActivity filter 6e463ed
          Action: "android.intent.action.VIEW"
          Category: "android.intent.category.DEFAULT"
          Scheme: "androgoat"
          Authority: "vulnapp": -1

  Non-Data Actions:
      android.intent.action.MAIN:
        e7fbe59 owasp.sat.agoat/.SplashActivity filter 892a204
          Action: "android.intent.action.MAIN"
          Category: "android.intent.category.LAUNCHER"

Receiver Resolver Table:
  Non-Data Actions:
      androidx.profileinstaller.action.SAVE_PROFILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 2794296
          Action: "androidx.profileinstaller.action.SAVE_PROFILE"
      androidx.profileinstaller.action.INSTALL_PROFILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 8045858
          Action: "androidx.profileinstaller.action.INSTALL_PROFILE"
      androidx.profileinstaller.action.SKIP_FILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 8d49db1
          Action: "androidx.profileinstaller.action.SKIP_FILE"
      androidx.profileinstaller.action.BENCHMARK_OPERATION:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 5ef3e17
          Action: "androidx.profileinstaller.action.BENCHMARK_OPERATION"

Permissions:
  Permission [owasp.sat.agoat.DYNAMIC_RECEIVER_NOT_E
```

Step 8 — Post-proof screenshot

```
screenshot

Screenshot saved as screenshot_02.png (role=post_proof)
```

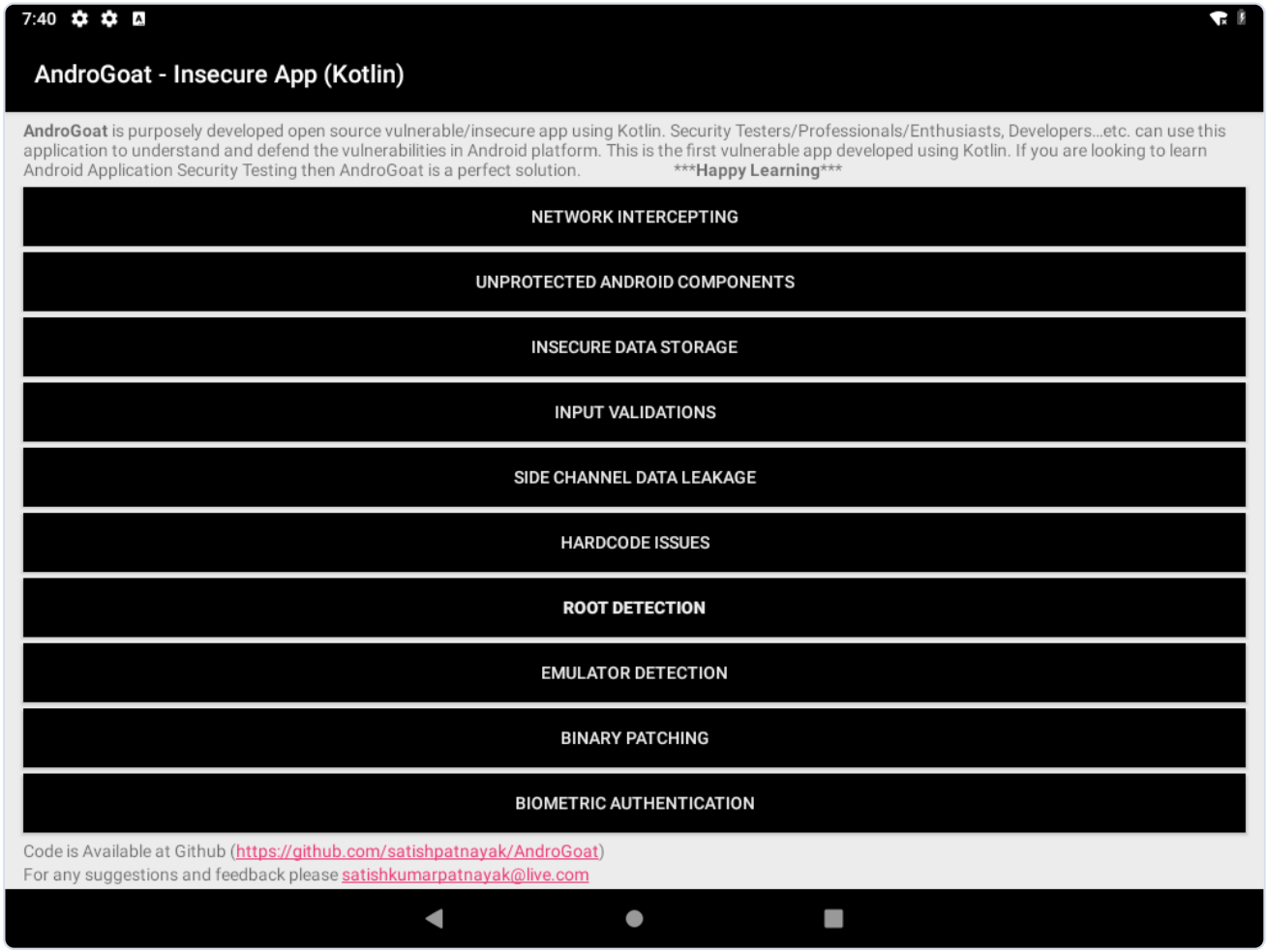
Command output — Launch target application

```
bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=2ms (0ms mobile, 0ms wifi, 2ms not connected)
```

Command output — Pull the installed AGoat APK and search its extracted contents for the hardcoded AWS access key string proving the secret is physically shipped in the shipped binary (MASVS-STORAGE-1 impact: credential exfiltration from any device with the APK).

```
/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==/base.apk: 1 file pulled, 0 skipped. 154.6 MB/s
(7100535 bytes in 0.044s)
/tmp/agoat_extracted/classes2.dex:ADD_FROZEN␣
ADD_SUCCESS␣ADD_TILE␣ADJUST_ROW_NUMBER_SKIP␣ADR␣ADR:␣AE␣AES␣AES/CBC/
PKCS7Padding␣AFFECTS_LOGO_MASK␣AGGREGATION_COUNT␣AHT␣AI013103decoder.java␣AI01320xDecoder.java␣AI0
1392xDecoder.java␣AI01393xDecoder.java␣AI013x0x1xDecoder.java␣AI013x0xDecoder.java␣AI01AndOtherAI
s.java␣AI01decoder.java␣AI01weightDecoder.java␣AIChatActivity.kt␣AIHint␣AKIAX56QKKOLPQ7G7ABC␣AL␣
ALIGN_CENTER␣ALIGN_HORIZONTALLY␣
ALIGN_NORMAL␣ALIGN_OPPOSITE␣ALIGN_VERTICALLY␣ALL␣ALLOWED_EAN_EXTENSIONS␣ALLOWED_LENGTHS␣
ALLOW_BINARY␣ALLOW_EDGE_LOCK␣ALLOW_SIZE_IN_UNSPECIFIED_SPEC␣ALLOW_THREAD_GAP_WORK␣ALL_FORMATS␣
ALL_OPERATOR    ALL_PAGES␣ALOG␣ALPHA␣ALPHABET␣ALPHABET_STRING␣
ALPHANUMERIC␣ALPHANUMERIC_CHARS␣ALPHANUMERIC_TABLE␣ALPHA_DISABLED␣ALPHA_DISABLED_LOW␣ALPHA_FRACTIO
N
```

Screenshot — Capture a screenshot of the device state after successful key extraction as supplementary visual evidence.



Command output — Bring target app to foreground

```
bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=5ms (0ms mobile, 0ms wifi, 5ms not connected)
```

Command output — Dump package components for this app only

```
Activity Resolver Table:
  Schemes:
      androgoat:
        4ea72a0 owasp.sat.agoat/.AccessControl1ViewActivity filter 6e463ed
          Action: "android.intent.action.VIEW"
          Category: "android.intent.category.DEFAULT"
          Scheme: "androgoat"
          Authority: "vulnapp": -1

  Non-Data Actions:
      android.intent.action.MAIN:
        e7fbe59 owasp.sat.agoat/.SplashActivity filter 892a204
          Action: "android.intent.action.MAIN"
          Category: "android.intent.category.LAUNCHER"

Receiver Resolver Table:
  Non-Data Actions:
      androidx.profileinstaller.action.SAVE_PROFILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 2794296
          Action: "androidx.profileinstaller.action.SAVE_PROFILE"
      androidx.profileinstaller.action.INSTALL_PROFILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 8045858
          Action: "androidx.profileinstaller.action.INSTALL_PROFILE"
      androidx.profileinstaller.action.SKIP_FILE:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 8d49db1
          Action: "androidx.profileinstaller.action.SKIP_FILE"
      androidx.profileinstaller.action.BENCHMARK_OPERATION:
        a395a1e owasp.sat.agoat/androidx.profileinstaller.ProfileInstallReceiver filter 5ef3e17
          Action: "androidx.profileinstaller.action.BENCHMARK_OPERATION"

Permissions:
  Permission [owasp.sat.agoat.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION] (514b2ff):
    sourcePackage=owasp.sat.agoat
    uid=10086 gids=null type=0 prot=signature
    perm=Permission{89418cc owasp.sat.agoat.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION}
    packageSetting=PackageSetting{64e715 owasp.sat.agoat/10086}

Registered ContentProviders:
  owasp.sat.agoat/androidx.startup.InitializationProvider:
    Provider{89cfa2a owasp.sat.agoat/androidx.startup.InitializationProvider}
  owasp.sat.agoat/.ContentProviderActivity:
    Provider{58b801b owasp.sat.agoat/.ContentProviderActivity}

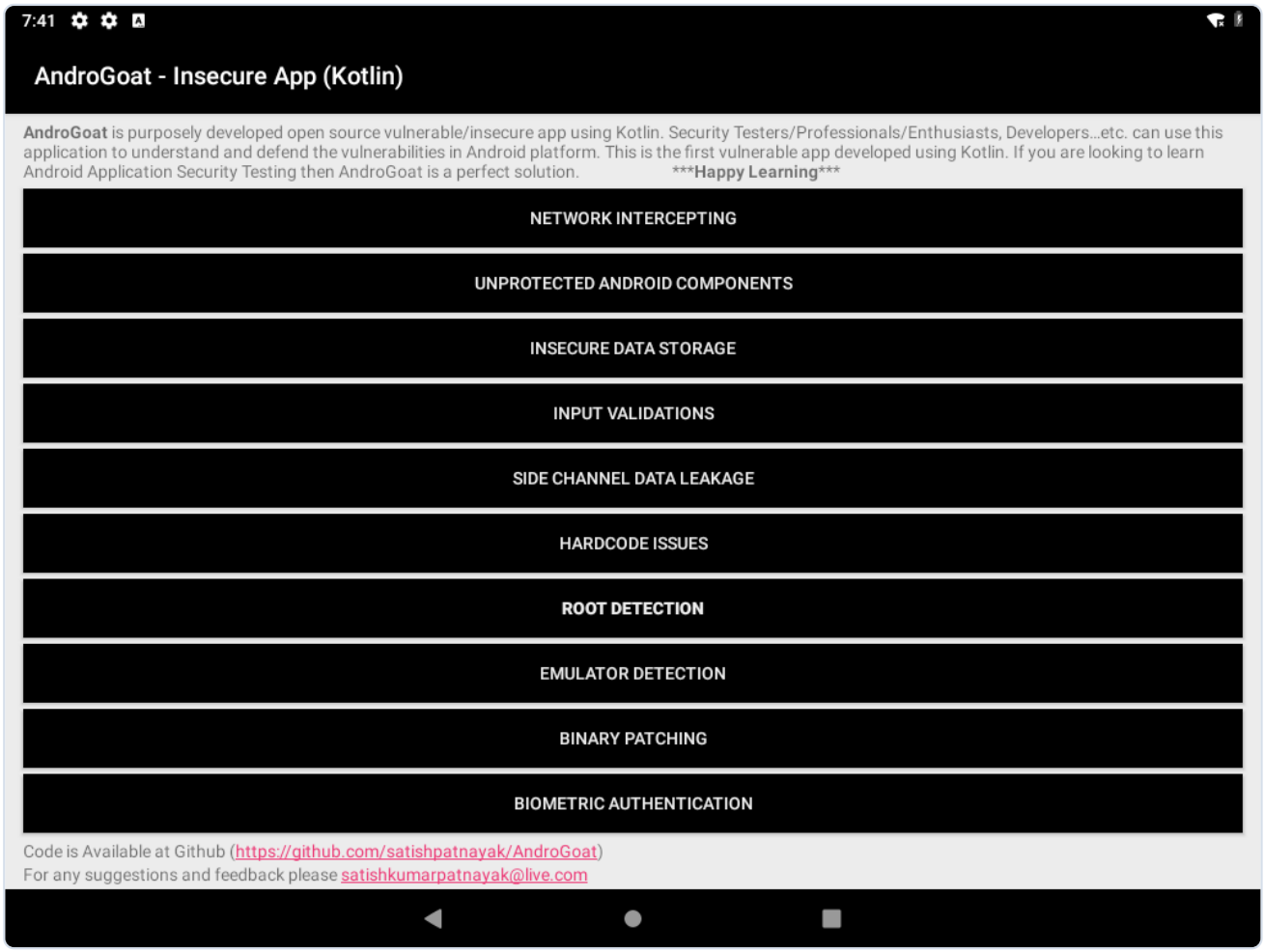
ContentProvider Authorities:
  [owasp.sat.agoat.provider.userpinsprovider]:
    Provider{58b801b owasp.sat.agoat/.ContentProviderActivity}
      applicationInfo=ApplicationInfo{d4ff5b8 owasp.sat.agoat}
  [owasp.sat.agoat.androidx-startup]:
    Provider{89cfa2a owasp.sat.agoat/androidx.startup.InitializationProvider}
      applicationInfo=ApplicationInfo{d4ff5b8 owasp.sat.agoat}

Key Set Manager:
  [owasp.sat.agoat]
      Signing KeySets: 24

Packages:
  Package [owasp.sat.agoat] (64e715):
    userId=10086
    pkg=Package{c39df91 owasp.sat.agoat}
    codePath=/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==
```

```
    resourcePath=/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==
    legacyNativeLibraryDir=/data/app/owasp.sat.agoat-mcROn1ANc3Bb9zUgwyVt7w==/lib
    primaryCpuAbi=null
    secondaryCpuAbi=null
    versionCode=1 minSdk=19 targetSdk=33
    versionName=1.0
    splits=[base]
    apkSigningVersion=2
    applicationInfo=ApplicationInfo{d4ff5b8 owa
```

Screenshot — Post-proof screenshot



Command output — Evidence gate

```
Dynamic verification: PARTIAL
Matched tokens: []
Reasons: ['No expected proof tokens from the finding appeared in exploit command output.']
```

DISCLAIMER

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.